SOFTWARE—PRACTICE AND EXPERIENCE, VOL. 10, 383–392 (1980)

UNIX with Satellite Processors

AMNON B. BARAK AND AMOS SHAPIR
Department of Computer Science, The Hebrew University, Jerusalem, Israel

SUMMARY

The steps necessary to extend the UNIX* time sharing system to a network which includes a central processor and a set of satellite processors is described. Software interfaces permit a program in the satellite processor to behave as if it were running in the central processor. Tasks are executed in parallel in several processors resulting in improved reliability and response time. The economics of such systems becomes more feasible with the reduction of the cost of CPUs and memories and the increasing demand for dedicated local computers.

KEY WORDS UNIX Distributed system Operating system

INTRODUCTION

Recent developments in computer hardware and the resulting price reductions affect the economics of computing and the ways computers may be used. One obvious trend is to incorporate several minicomputers and microcomputers in a network with resource sharing under the control of a central operating system. A simple method of constructing such a system is to extend an existing operating system from a central processor based system, called here a master controller (MC) to a set of satellite processors (SP). The main advantage of such a configuration is that a large computing system is extended to many attached processors, giving each satellite processor access to the MC resources while retaining the flexibility of a dedicated local computer. Since the cost of peripherals in many cases exceeds the cost of a CPU, the extended system may also provide an economical solution to an increasing demand for computing power by allowing the MC to support a pool of peripherals for the possible use of many SPs.[5] This could be done by using a simple allocation procedure in which all system tasks are executed in the MC, while user programs are executed in the SPs.

This paper gives a detailed description of the steps necessary to construct a system of satellite processors centred around UNIX.[6]

UNIX is a multi-user, time-sharing, interactive operating system. It is interrupt driven, and is dependent on a single processor for execution. All the activities in this system are carried out through processes; the CPU is always executing on behalf of a certain process.

A UNIX process is an execution of a program which originally resided in a file. During execution, the program resides in memory, as a part of an execution environment of the process. Other assets of the process environment may be additional memory, access to disk files and certain parts of the system's data.

* UNIX is a trademark of Bell Laboratories.

0038–0644/80/0510–0383$01.00
© 1980 by John Wiley & Sons, Ltd.
Received 13 July 1979
Revised 10 December 1979

There is no 'central process' in UNIX for supervising activities. The system consists of a set of subroutines, which are activated by users' processes or by hardware interrupts. In addition to requests for resources, a UNIX process can create a new process, execute a file or terminate itself. There are also several means of interprocess communication.

Scheduling of time and space among processes in UNIX is done in a simple manner. Each ready-to-run process enters the execution queue; the executing process runs until it requests a resource that is not available immediately, or until it is interrupted by hardware. After dealing with the interrupt or the request, the stopped process sets in motion the next process on the queue. A waiting process may be swapped out to the disc, leaving in memory only the essential part of its environment.

In UNIX, the interface between a user program and the operating system is accomplished by means of system calls. This concept was extended in our system to the SP; however, since the SP does not have its own resident system it can only trap the call and pass it with its parameters to the supervisory system in the MC. A monitor, which is an MC process, running on behalf of each SP, executes the system call and passes the results back to the SP. Control is then returned to the user program in the SP. In our implementation each SP executes a single task which is assigned to it by the MC. It has access to all the supervisor's resources, such as the file system and the peripherals, by means of system calls.

Our implementation consists of three levels. The first level, described in more detail in the section *The Satellite Kernel,* is the kernel which is the minimal part of the SP resident system needed to load and run tasks from the MC. Each satellite processor has a kernel which consists of an interrupt handler, interprocessor communication package and a general request handler to communicate all system requests from the current user task to the supervisor level.

The second level is the monitor, it resides in the MC processor and handles sstem calls on behalf of the user task in the SP. The monitor is described in the section *The Monitor.*

The third level, the supervisor, which is the uppermost level of the MC system, is described in the section *The Supervisor.* It consists of a modified version of UNIX. It is responsible for resource allocation, file management, memory management, I/O monitors and scheduling of task execution in the SPs.

A brief description of the hardware configuration is given in the next section and some comments about our implementation are given in the section *Software Implementation.*

ORGANIZATION AND CONFIGURATION OF THE HARDWARE SYSTEM

The configuration of the hardware system is a star architecture consisting of two logical levels, connected by parallel communication devices such as a direct memory access, or a dual port memory.

The first level is the user level. It consists of two or more processors, called satellite processors, each with its own local memory and communication devices. Each satellite processor is connected to the second level master controller processor through its communication device.

The master–satellites scheme enables the system to relieve the satellite processors of most of the system-managing and I/O chores, which are carried out by the MC processor; thus the satellite processors are free to execute computational tasks which require fast response time and relatively large amounts of primary memory.

The particular hardware configuration described here consists of a DEC PDP–11/45 central master controller computer,[2] and a PDP–11/10 satellite processor. An additional number of satellite processors which belong to the same family of computers such as the LSI–11/23 are scheduled for integration into the system.

THE SATELLITE KERNEL

The kernel is the minimal part of the satellite operating system needed to load and run tasks from the MC. The kernel consists of three functions: (1) Interrupt handlers—to deal with hardware and software interrupts; (2) Interprocessor communication package; (3) A system-calls handler—to deal with system requests of the current task. These functions are implemented across all the processors, including the MC. This enables the system to dispatch tasks to any of the satellite processors, or execute them on the MC, without any changes.

A copy of the kernel resides in each of the satellite processors, as well as in the MC, and provides services for the 'upper-level routines' (i.e. users' tasks on the SPs, and the supervisor on the MC). The differences between the various satellite kernels are due to local peripherals, which might be resident in one processor and not in the others.

The interrupt handler

The interrupt handler is necessary in each processor to allow processing of interrupts, which are caused asynchronously by local devices, or by software (traps). The interrupt-handler deals with the source of the interrupt in real time seemingly without interfering with the user's task currently executing.

The PDP–11 architecture provides a powerful tool for handling interrupts through the mechanism of automatic priority interrupt handling.[2] A reserved area in memory is used as a stack. When an interrupt occurs, the program counter (PC) and the program status word (PSW) registers are pushed onto the stack, and new values, taken from an interrupt vector, are then used, according to the source of the interrupt. The interrupt vector supplies the address of the handling routine and the appropriate PSW. These are loaded, causing a branch to the interrupt handling routine.

The PSW contains, among other information, the processor's priority during execution (between 0 and 7). Devices whose priority is not higher than that of the processor, cannot interrupt until the current priority is lowered. Thus the new PSW, established by an interrupt-service routine, can block all interrupts from any device until the current one is dealt with; devices of higher priority can interrupt and their interrupts may be serviced. Software interrupts have the highest priority, and cannot be blocked. This mechanism provides a hardware solution to the mutual exclusion problem, as the processor priority may be raised explicitly to block out all interrupts during the execution of any critical section.

When the interrupt routine terminates, previous values of the PC and PSW are restored from the stack. Access to the saved values of the PC and PSW is easy even for an interrupt handler routine written in a high-level language (e.g. C[4]), thus the PC and

PSW (and any other registers saved in the stack) are available as parameters of the interrupt-handler routine. If these values were changed by the interrupt-handler, the return may cause an off-line branch in the user program.

The communication package

All the communication between the satellite and the MC is done by the communication package, which resides in both the MC and the satellite kernels. The transmission is completely asynchronous, using interrupts on the communication line. An incoming message initiates an interrupt (in the receiver), which then activate the interrupt-handler followed by activation of the receiving procedure. Messages are transferred by the initiation of the sending side only, thus allowing the MC to control the execution of tasks on the satellite processors.

The package consists of two main primitives: a sending routine, (called 'send'), and a receiving routine, which is activated by an interrupt from the incoming message. A third routine, 'receive', is used to wait for an incoming message. The routines 'receive' and 'send' are functionally equivalent to Hansen's basic primitives WAIT and SEND.[3] Since communication is carried out over a relatively short distance, and always between the MC and the SPs, a simple protocol is used.

The following messages are currently implemented: (1) Synchronous communication—for system calls; (2) Asynchronous loading—for loading segments of code; (3) Transfer of control—to start the execution of a loaded task; (4) Exit—the current task is stopped; (5) Load onto stack—the SP stack area is overwritten by the message. These messages enable the MC to control the execution of tasks. Although all types are recognized on both sides of the line, messages from the satellite kernel to the MC are always of the first type. The interrupt-handler routine's capability of changing its own return address is used here to execute transfers of control.

The system-calls handler

System-calls are requested by the user tasks. They enable these tasks to interact with the system's environment. Such interactions are needed for file handling, communication between users, etc.

All requests to the system from a user task are carried out by execution of the 'trap' instruction in the SP. This instruction is a software-generated interrupt, which activates the trap handler. The 'trap' instruction contains the call number, and parameters to the system-call immediately follow the instruction. These parameters are usually pointers to other parts of the memory, where data for the request is stored, or where the reply is to be returned.

The request number is used as a pointer which may be one of three possibilities: (1) Illegal request (not all the requests are meaningful): an appropriate error condition is returned; (2) Local request, executed directly in the satellite processor; (3) Non-local request is sent to the MC for execution.

If the request cannot be executed locally, a pointer is set to an entry in the system-calls table (SCT). Each entry in this table contains the number of parameters of the call, and their usage. From this information a message which contains all the data that is needed for the execution of the request is constructed.

The message is broken down by the monitor, which also contains a copy of the SCT table. It reconstructs and executes the system-call, then sends back a reply message, containing the new values of the parameters, followed by the requested data. Finally the

system-calls handler copies the data into the requested address, and returns the control to the user's task.

THE MONITOR

The monitor is a process which resides in the MC processor. It handles system calls in the MC on behalf of a user task. Thus it constitutes the only link between the supervisor in the MC processor, and the user's task in the SP. The monitor is responsible for loading a task into the SP and servicing it during execution. It does so in accordance with the supervisor's orders in the MC.

All control operations of the monitor related to a task—loading, starting and stopping—are accomplished by sending control messages. Each message from the SP to the monitor is treated as a system request. The monitor executes system requests by reconstructing the caller's relevant environment at the time of the request. Space is then allocated for data areas needed by pointer parameters. Note that no constant buffers are used, since the size of the data space needed for the call is known only when it is accepted as a message.

The request itself is made by performing the 'indirect' system-call, which is used in UNIX to execute 'trap' instructions that are not in the caller's instruction space, but are given as data. After the request is carried out, a reply message is sent back with new values of the parameters, registers and data areas.

There are three system calls which must be given special treatment by the monitor. These are the request to terminate execution, called 'exit'; the request to execute a set of instructions from a file, called 'exec' and the request to form a new process, called 'fork'.

When an 'exit' request is received, the monitor notifies the supervisor that the SP is free, and terminates its activity. The 'exec' request causes the monitor to load the named file into the SP, and execute it as a new task.

The 'fork' request is the only way to create a new process in UNIX: the newly created ('child') process is a copy of its creator (the 'parents'). The 'child' resumes its execution at the instruction following the 'fork' request, while the 'parent' skips this instruction when it resumes execution. The only difference between the 'parent' and the 'child' processes is the return address, and the fact that the value returned by the call to the 'parent' is the process-id of the 'child'.

During the execution of the 'fork' request, resources (such as memory and system buffers) are allocated to the new process. The new process may request an 'exec' immediately. Thus any process may execute any (executable) file, and the system's job consists only of fulfilling the processes' requests.

The 'fork' request causes the monitor to create a process which is a new monitor. The new monitor requests the supervisor to allocate an SP (the only resource not supplied by UNIX) for the new task. Once an SP is allocated, a copy of the user's task is transferred from the 'old' SP through the monitors into the 'new' SP. Thus the user's task is also duplicated, and execution of both tasks is resumed, the new task simulating a return from the 'fork' request.

THE SUPERVISOR

The supervisor is the uppermost level of the system. It allocates the MC processor resources to users' processes, and schedules their execution. The resources include

CPU time, memory in the MC processor, secondary memory (and other devices) usage, and satellite processors.

The supervisor recognizes user tasks by means of their monitors, each monitor represents a single user task, which has to be run in one of the SPs. Since monitors are regular user-level processes, almost all the supervising activities are carried out by the host UNIX system.

The only resources that cannot be managed directly by UNIX are satellite processors. These resources are managed in our system by the scheduler. The scheduler communicates with the monitors through a small shared data area, which contains the monitor-processor table (MPT) of all the existing monitors and their SPs. Each new monitor writes its own process-id number in the MPT, and waits. The scheduler reads the MPT periodically and allocates (upon availability) an SP for each waiting monitor. It then reactivates the monitor, which starts by loading a task into the SP.

An allocated SP remains under the monitor's control until the user's task terminates. When a user task terminates, the corresponding monitor erases its own process-id number from the MPT, and signals the scheduler that the SP is free and may be reallocated.

All communications between the MC and the SPs are carried out by the monitors. The supervisor is unaware of users' tasks executing in the SPs and it deals only with their monitors. The supervisor can, however, request the monitor to terminate execution, which results in terminating the execution of a user's task in the SP, and freeing it for further allocation.

SOFTWARE IMPLEMENTATION

In this section we give some details about the implementations of our system. It is assumed that the reader is familiar with the UNIX system.[6] The fundamental idea is to extend the single processor system for operation in a multiple processor environment. The extended system operates as a user-level process and uses UNIX features whenever possible, in order to allow resource sharing among user tasks. Resources which are not managed by UNIX, i.e. satellite processors and communication, are managed by user-level processes.

The system consists of two parts: a resident part and a non-resident part. The resident part includes the kernel in the SP, and UNIX, the scheduler and the command-line-interpreter (CLI) in the MC. The non-resident part includes the monitors, one for each executing user task.

The scheduler runs as a UNIX user process. It schedules SPs among users tasks. The CLI is another user process, which is a modified version of UNIX's shell;[1] however, instead of directly executing a named file, it executes a monitor. Each monitor is a user process which is the user task's image under UNIX. It is 'born' and 'dies' with the user's task, and passes all its requests to UNIX as if they were initiated by it.

A user task is a program which originally resides as a UNIX file. During execution, it is loaded by the monitor into the SP's memory. Since hardware memory management enables UNIX to allocate a full 64KB of virtual memory for each user, the compilation and link routines create executable files which start at virtual address 0, and are not relocatable. A special compilation and linking routine is used for SPs without a memory

management unit. Lycklama and Christensen[5] used a similar approach. However, in our implementation executable files do not contain parts of the satellite kernel.

Interface with the human operator is done by the CLI. Its mode of operation is similar to that of UNIX's shell; however, instead of issuing an 'exec' system call for each command, it creates an executable file that contains the 'exec' call, then forks a monitor to carry it out. The CLI is the only user task that runs in the MC processor without a monitor. This is done for efficiency, but in principle the CLI could be implemented as a user task in one of the SPs.

Satellite kernel implementation

The satellite kernel is implemented as a set of routines which are activated by interrupts. Its size is approximately 5·5KB. User tasks which run in the SP do not have a direct access to the kernel routines except through software-generated interrupts ('traps').

Software interrupts are caused by requesting a 'trap' or 'emt' (emulated trap) instructions, or by an error, i.e. illegal instruction, illegal address (bus error), I/O trap or power failure. The 'trap' (UNIX's 'sys') instruction invokes the system-call handler, while an 'illegal instruction' activates the extended instruction set (EIS) emulation routine within the kernel, which tries to emulate the offended instruction. In all the other cases an error message is printed at the console.

The start-up sequence sets the hardware stack pointer to the top of memory, enables interrupts from local devices, then clears all unused memory. Clearing of memory and resetting the stack pointer is done after the termination of each user's task. Between executions of tasks, the SP waits for an interrupt in an idle loop.

Communication implementation

The communication package is almost identical on both the satellite kernel and the monitors. The package consists of the following routines:

1. a sending routine;
2. a synchronous receiving routine;
3. an asynchronous receiving routine;
4. line drivers which are used by the other routines for I/O operations.

The actual I/O operations are carried out by the drivers, which transfer a specified number of bytes from/to a specified address, and are transparent to the sender/receiver. In the monitor this is done by a single UNIX 'read' or 'write' system call.

The asynchronous receiving routine is activated by an incoming message. In the monitor it is called by the synchronous receiving routine, and in the satellite kernel—by the data-line interrupt-handler.

A local variable holds the address into which the message body will be read. It is set according to the message type as follows:

1. Synchronous communication—the message is transferred to an address specified by the receiving routine. This is the only type recognized by the monitor version of the communication package.
2. Loading—the address is contained in the first word of the message body.
3. Transfer of control—the message body consists of a single word, which is read into a global variable used by the interrupt handler as the return address. Upon returning from the interrupt, execution will resume at the address received in the message.

4. Exit—the message body is empty, and 'exit' call is performed. This causes clearing of memory and waiting for the next task.
5. Load into stack—the message is received into a temporary buffer. At the end of transmission, the buffer is transferred to the top of the stack. This is done to prevent destruction of local variables which reside in the stack. The return address is contained in the new stack.

System calls implementation

Handling of system calls by the satellite kernel is an emulation of UNIX's handling of these calls. Using this method a user task which executes a system call gets the same results it would have received from UNIX.

If a system call cannot be executed locally, the handler looks it up in the non-local calls table. Each entry in the table contains the number of the call's arguments, and flags that indicate which arguments are pointers, and which arguments should be sent over to UNIX.

From this information a message is constructed. It includes: The call number and arguments; user's registers r0 and r1, and the arguments which are necessary for the execution of the call. The message is sent to the monitor for execution, and the kernel waits for its reply. Calls implemented in this manner are: read, write, open, close, creat, link, unlink, chdir, time, chmod, stat, seek, getpid, getuid, fstat, stty, gtty, dup, getgid and empty.[7]

Local calls are executed, as in UNIX, by performing a local function directly. Calls implemented locally are: exit, fork, exec, break, sleep and csw.[7]

Three of the calls affect the layout of processes in the system, and require intervention of the supervisor, via the monitor:

1. Exit—a message which contains only the call number is sent to the monitor, which releases its hold of the SP and exits. The satellite kernel clears the user's memory and waits for the next task.
2. Fork—the call number is sent to the monitor which executes a 'fork' and sends back the 'child' monitor's process-id. If this id is valid, the user's memory and stack areas are sent to the monitor, which passes them on to its 'child' for loading into another SP. The current user becomes the 'parent' task. The 'child' task resumes execution in the other SP.
3. Exec—a message which consists of the call number and its first argument is sent to the monitor. The rest of the arguments are copied from the user's memory into a temporary buffer. The first argument is a file name, which must be compiled for execution in an SP. The monitor sends this file back to be loaded into the user's memory. The unused part of the user's memory is cleared and the arguments are copied into the stack. Execution then resumes at the beginning of the user's memory.

Monitor implementation

The monitor is a UNIX user-process which is an image of the user's task within the framework of UNIX. Its size is approximately 2KB. One monitor is created for each user task, it is 'born' with the task and 'dies' with it. The monitor executes the same system calls the user task would have executed if it was run under UNIX, thus letting UNIX 'think' that the user's task is executed in the MC processor.

A monitor, like any other UNIX process, is created by the 'fork' and 'exec' system calls of its 'parent' process. the 'parent' in this case is a command-line interpreter or another monitor. The newly created monitor receives as an argument the name of a file which contains a copy of the user's task. The structure of this file is similar to that of a UNIX executable file.[6] First the monitor requests an SP number and waits for the scheduler to allocate it. Once an SP number is allocated, the monitor uses this number to open the SP's special-device file for communication. After establishing communication with the SP, the monitor transfers the copy of the dumped file into the SP memory including text, date and the user's stack. Execution of the user's task is then resumed in the SP.

During the execution of a task, its monitor waits for an incoming message and each message is treated as a system call. The 'trap' instruction and its arguments are reconstructed in a local array, and executed. Data areas produced by the call are appended to the message buffer, and the message is sent back to the SP.

The three system calls, 'exit', 'fork' and 'exec', cause the monitor to release, allocate or load an SP respectively as described above.

Scheduler implementation

Scheduling of SPs usage among users tasks is the only MC supervisory activity that is not supported by UNIX. The scheduler is a user-level process, and uses UNIX interprocess communication facilities to control the monitors. Its size is approximately 1/2KB.

Exchange of data between the scheduler and the monitors is done through a shared-data area which contains a table of all the active monitors' ids and their respective SPs' numbers. Each monitor enters its id in the table when it is 'born', and erases its id when it 'dies'. The scheduler updates the SPs' numbers in the table according to the monitors' requests and the availability of SPs.

CONCLUDING REMARKS

In this paper we described an exercise in allocation of user tasks to a set of satellite processors. The basic idea was to use a powerful system such as UNIX, which normally runs on a single processor and extend it to a network of processors under a central control.

Experimental execution of jobs on our system has shown that parallel execution of CPU-bound tasks on two processors takes up to 60 per cent of the time it would have taken on a single processor. On the other hand, tasks that require many system calls (such as file listing) are much slower when executed in an SP, due to communication overhead. Communication overhead is also an important factor which restricts the maximum number of SPs the system can support.

The system is currently in use in our department. It serves mainly to off-load the 'slow' devices such as the graphical display terminal from the central computer.

This project is the first phase of a design of a UNIX-based distributed system. The second phase is intended to make system calls and drivers functionally independent of the UNIX kernel, thus allowing the utilization of a peripheral device or a special system call by an SP.

REFERENCES

1. S. R. Bourne, 'The UNIX shell', *The Bell System Tech. Journal,* **57**, 6, part 2, 1971–1990 (1978).
2. *DEC PDP–11 Processor Handbook* (1975).
3. P. Brinch Hansen, 'Structured multiprogramming', *Comm. ACM,* **15**, 7, 574–578 (1972).
4. B. W. Kernighan and D. M. Ritchie, *The C Programming Language,* Prentice-Hall, Inc., Englewood Cliffs, N.J., 1978.
5. H. Lycklama and C. Christensen, 'A minicomputer satellite processor system'. *The Bell System Tech. Journal,* **57**, 6, part 2, 2103–2113 (1978).
6. D. M. Ritchie and K. Thompson, 'The UNIX time-sharing system', *Comm. ACM,* **17**, 7, 365–375 (1974). Also *The Bell System Tech. Journal,* **57**, 6, part 2, 1905–1929 (1978).
7. K. Thompson and D. M. Ritchie, *UNIX Programmer's Manual,* 6th edn., Bell Telephone Lab., Murray Hill, N.J., 1975.